

Problem1

Query 1

List all students who are registered for a course section and are also on the waiting list for another course section in the same semester.

Involved Entity Sets:

Students (strong entity)

Sections (weak entity)

Involved Relationship Sets:

Registering (Students - Sections)

waits_on (Students - Sections)

Query 2

Find all GTAs who are assigned to work for a course section but have not received an evaluation rating yet.

Involved Entity Sets:

GTA (subclass of Students)

Sections (weak entity)

Students (superclass of GTA)

Involved Relationship Sets:

Assigned_to (GTA - Sections)

ISA (Students - GTA)

Problem2

Requirements that cannot be captured by the given ER diagram:

Requirement 1: Conditional constraints across multiple relationship sets

Some constraints depend on conditions involving more than one relationship set, which cannot be enforced by ER diagrams alone.

For example, a student cannot be both registered for and on the waiting list of the same course section at the same time.

This restriction involves two relationship sets:

Registering (Student - Section)

waits_on (Student - Section)

An ER diagram cannot express such mutual exclusion constraints across relationships. These constraints must be enforced using application logic or database-level constraints.

Requirement 2: No more than 10 students are allowed to be on the waiting list for the same course section at the same time.

Standard ER diagrams cannot express such specific numerical limits (e.g., "at most 10").

Therefore, this constraint cannot be fully captured by the ER diagram and must be enforced at the database or application level.

Requirement 3: GTA work termination constraint

GTA's are not allowed to terminate their work before the end of the semester.
Standard ER notation cannot represent time-based or lifecycle constraints on relationships.
Therefore, this requirement cannot be captured in the ER diagram and must be enforced procedurally.

Problem3

Course(dept: string, course#: string)

Primary Key: (dept, course#)

Candidate Key: (dept, course#)

No Foreign Key

Professor(pssn: string, pname: string)

Primary Key: pssn

Candidate Key: pssn

No Foreign Key

Student(sssn: string, sname: string)

Primary Key: sssn

Candidate Key: sssn

No Foreign Key

GTA(sssn: string, salary: real)

Primary Key: sssn

Candidate Key: sssn

Foreign Key: sssn → Student(sssn)

Sections(dept: string, course#: string, section#: string, taught_by_pssn: string)

Primary Key: (dept, course#, section#)

Candidate Key: (dept, course#, section#)

Foreign Key:

(dept, course#) → Course(dept, course#)

taught_by_pssn → Professor(pssn)

Can_teach(pssn: string, dept: string, course#: string)

Primary Key: (pssn, dept, course#)

Candidate Key: (pssn, dept, course#)

Foreign Key:

pssn → Professor(pssn)

(dept, course#) → Course(dept, course#)

Prereq_of(dept:string, course#:string, prereq_dept:string, prereq_course#: string)

Primary Key: (dept, course#, prereq_dept, prereq_course#)

Candidate Key: (dept, course#, prereq_dept, prereq_course#)

Foreign Key:

(dept, course#) → Course(dept, course#)

(prereq_dept, prereq_course#) → Course(dept, course#)

Registering(sssn: string,dept: string,course#: string,section#: string)

Primary Key: (sssn, dept, course#, section#)

Candidate Key: (sssn, dept, course#, section#)

Foreign Key:

sssn → Student(sssn)

(dept, course#, section#) → Sections(dept, course#, section#)

Transcript(sssn: string,dept: string,course#: string,section#: string,grade: string)

Primary Key: (sssn, dept, course#, section#)

Candidate Key: (sssn, dept, course#, section#)

Foreign Key:

sssn → Student(sssn)

(dept, course#, section#) → Sections(dept, course#, section#)

Waits_on(sssn:string,dept:string,course#:string,section#: string,wait_position:string)

Primary Key: (sssn, dept, course#, section#)

Candidate Key: (sssn, dept, course#, section#)

Foreign Key:

sssn → Student(sssn)

(dept, course#, section#) → Sections(dept, course#, section#)

Assigned_to(gta_ssn:string,dept:string,course#:string,section#: string,rating: integer)

Primary Key: (gta_ssn, dept, course#, section#)

Candidate Key: (gta_ssn, dept, course#, section#)

Foreign Key:

gta_ssn → GTA(sssn)

(dept, course#, section#) → Sections(dept, course#, section#)

Relation	Attributes (with domain/d ata type)	Primary key	Foreign key(s)	Candidate key(s)
Course	dept VARCHAR 2(10), course# VARCHAR 2(10)	(dept, course#)	—	{{dept, course#}}
Professor	pssn	pssn	—	{pssn}

	CHAR(9), pname VARCHAR 2(50)			
Student	sssn CHAR(9), sname VARCHAR 2(50)	sssn	—	{sssn}
GTA	sssn CHAR(9), Salary NUMBER(10,2)	sssn	sssn→Student(sssn)	{sssn}
Sections	dept VARCHAR 2(10), course# VARCHAR 2(10), section# CHAR(2), taught_b y_pssn CHAR(9)	(dept,cour se#, section#)	(dept,course#)→Course(dept,cour se#);taught_by_pssn → Professor(pssn)	{{dept,cou rse#, section#}}
Can_teach (Professor–Course)	pssn CHAR(9), dept VARCHAR 2(10), course# VARCHAR 2(10)	(pssn, dept, course#)	pssn→Professor(pssn); (dept, course#) → Course(dept, course#)	{{pssn, dept, course#}}

Prereq_of (Course–Course recursive)	dept VARCHAR 2(10), course# VARCHAR 2(10) prereq_dept VARCHAR 2(10), prereq_course# VARCHAR 2(10)	(dept, course#, prereq_dept, prereq_course#)	(dept,course#)→Course; (prereq_dept,prereq_course#)→ Course	{{dept, course#, prereq_dept, prereq_course#}}
Registering (Student–Section)	sssn CHAR(9), dept VARCHAR 2(10), course# VARCHAR 2(10), section# CHAR(2)	(sssn, dept, course#, section#)	sssn→Student;(dept,course#,section#)→ Section	{{sssn, dept, course#, section#}}
Transcript(Sections- Students,has grade)	sssn CHAR(9), dept VARCHAR 2(10), course# VARCHAR 2(10), section# CHAR(2), grade VARCHAR 2(2)	(sssn, dept, course#, section#)	sssn→Student;(dept,course#,section#)→ Section	{{sssn, dept, course#, section#}}
Waits_on(Students- Sections,has wait-position)	sssn CHAR(9), dept VARCHAR 2(10), course# VARCHAR	(sssn, dept, course#, section#)	sssn→Student;(dept,course#,section#)→ Section	{{sssn, dept, course#, section#}}

	2(10), section# CHAR(2), Wait-positi tion CHAR(2)			
Assigned_to(GTA-Se ctions,has rating)	gta_ssn CHAR(9), dept VARCHAR 2(10), course# VARCHAR 2(10), section# CHAR(2), rating NUMBER(3)	(gta_ssn, dept, course#, section#)	gta_ssn→GTA(ssn);(dept,course# ,section#)→ Section	{(gta_ssn, dept, course#, section#)}